

**YENEPOYA INSTITUTE OF ARTS,
SCIENCE,
COMMERCE AND MANAGEMENT**

Project –High Level Design

on

**Visual Threat Hunting for Manufacturing
Networks**

Student Name: Ahmed Jasim Abdullah

Industry Mentor:
Mr.Sumith
Kumar Shukla

Guided
by:Mr
Vikyath
Shetty

Table of Contents

- 1. Introduction**
 - 1.1 Scope of the document**
 - 1.2 Intended Audience**
 - 1.3 System overview**
- 2. System Design**
 - 2.1 Application Design**
 - 2.2 Process Flow**
 - 2.3 Information Flow**
 - 2.4 Components Design**
 - 2.5 Key Design Considerations**
 - 2.6 API Catalogue**
- 3. Data Design**
 - 3.1 Data Model**
 - 3.2 Data Access Mechanism**
 - 3.3 Data Retention Policies**
 - 3.4 Data Migration**
- 4. Interfaces**
- 5. State and Session Management**
- 6. Caching**
- 7. Non-Functional Requirements**
 - 7.1 Security Aspects**
 - 7.2 Performance Aspects**
- 8. References**

1. Introduction

1.1 Scope of the Document

This document presents the High Level Design (HLD) for the Manufacturing Threat Hunter platform. It describes the system architecture, component structure, data flows, interface specifications, security considerations, and non-functional requirements to serve as the architectural blueprint for development and evaluation.

1.2 Intended Audience

- **Software Developers and ML Engineers**
- **Project Reviewers and Evaluators**
- **Academic Supervisors and Industry Mentors**
- **Industrial Network Security Analysts**
- **System Architects**

1.3 System Overview

Manufacturing Threat Hunter is an AI-powered cybersecurity dashboard purpose-built for industrial manufacturing networks. It continuously monitors network traffic to detect, analyze, and visualize security threats using an ensemble of four Machine Learning models — Isolation Forest, Local Outlier Factor (LOF), K-Means Clustering, and Random Forest — combined with a Heuristic Rule Engine. The platform features:

- **Real-time Live Traffic Simulation Mode**
- **Predictive next-hour traffic forecasting using Linear Regression**
- **Automated PDF security report and CSV threat log generation**
- **Premium dark-themed Streamlit dashboard optimized for SOC environments**

2. System Design

2.1 Application Design

The application is designed as a modular, in-memory processing pipeline with no persistent backend database. All computation occurs in-memory using Pandas/NumPy, with ML model state persisted via Joblib serialization.

Layer	Technology	Responsibility
Presentation Layer	Streamlit (Python)	Web UI: sidebar controls, KPI cards, tabbed analytics, live simulation, report downloads
Business Logic Layer	Python Modules (utils/)	Data preprocessing, ensemble ML detection, threat scoring, forecasting
Visualization Layer	Matplotlib, Seaborn	7 security-focused charts rendered server-side as static images
Reporting Layer	FPDF, Pandas CSV	PDF threat summary report and full CSV export for SIEM integration
Persistence Layer	Joblib (.pkl files)	Serialized ML models and scaler stored in models/ directory
Styling Layer	Custom CSS	Industrial dark-mode theme applied to Streamlit UI

2.2 Process Flow

[Process Flow Diagram should be inserted here showing the following sequence:]

- **Step 1 – Data Ingestion:** User uploads CSV or system generates synthetic network log data
- **Step 2 – Schema Validation:** Required columns verified (timestamp, device_ip, protocol, bytes_sent, bytes_received, connections)

- **Step 3 – Preprocessing:** Imputation, feature engineering, 99th-percentile outlier capping
- **Step 4 – Feature Scaling:** StandardScaler applied to feature matrix
- **Step 5 – Heuristic Rules:** 4 attack rules applied; pseudo-labels generated for Random Forest
- **Step 6 – ML Ensemble:** Isolation Forest, LOF, K-Means, Random Forest trained and evaluated
- **Step 7 – Threat Scoring:** Composite score (0-100) calculated; Threat Level assigned per record
- **Step 8 – Forecasting:** Linear Regression predicts next-hour traffic; SPIKE LIKELY flag set if > 95th percentile
- **Step 9 – Dashboard Rendering:** KPI metrics, 7 charts, AI insights tab populated
- **Step 10 – Reporting:** PDF/CSV generated on demand; Live Mode loops back to Step 5 each cycle

2.3 Information Flow

Stage	Input	Output	Module
File Upload / Synthetic Gen	CSV or integer row count	Raw DataFrame	preprocessing.py
Validation	Raw DataFrame	Validated DataFrame or error	preprocessing.py
Cleaning & Feature Eng.	Validated DataFrame	Feature-enriched DataFrame	preprocessing.py
Outlier Capping	Feature DataFrame	Capped DataFrame	preprocessing.py

Stage	Input	Output	Module
Feature Scaling	Capped DataFrame	Scaled NumPy ndarray	detector.py
Heuristic Rules	Feature DataFrame	Rule-labeled DataFrame (pseudo-labels)	detector.py
ML Ensemble Training	Scaled array + pseudo-labels	4 trained model objects	detector.py
ML Prediction	Scaled array	Anomaly flags per model per row	detector.py
Threat Scoring	Flagged DataFrame	Scored DataFrame with Threat Level	detector.py
Forecasting	Hourly aggregated traffic	Next-hour prediction + SPIKE flag	detector.py
Visualization	Scored DataFrame	7 Matplotlib/Seaborn figures	visualizer.py
Reporting	Scored DataFrame + stats	PDF bytes + CSV string	report.py

2.4 Components Design

Component	File	Type	Description
Main Application	app.py	Orchestrator	Streamlit entry point; UI layout, simulation

Component	File	Type	Description
			loop, pipeline coordination
Preprocessor	utils/preprocessing.py	Processing Module	Data cleaning, feature engineering, outlier capping, synthetic data generation
Threat Detector	utils/detector.py	ML + Rules Module	ThreatDetector class: 4 ML models, heuristic rules, threat scoring, forecasting
Visualizer	utils/visualizer.py	Visualization Module	7 Matplotlib/Seaborn security charts in dark theme
Reporter	utils/report.py	Reporting Module	PDF generation via FPDF; CSV export for SIEM integration
Data Directory	data/	Storage	Input CSV network log files
Models Directory	models/	Storage	Joblib-serialized .pkl model files (isolation_forest, lof, kmeans, random_forest, scaler)

2.5 Key Design Considerations

- **Ensemble Detection:** Four complementary ML models (unsupervised, clustering, supervised) minimize false positives that any single model would produce

- **Automated Labeling:** Random Forest trained on rule-generated pseudo-labels solves the unlabeled data problem common in network security datasets
- **Real-Time Capable:** Live simulation loop demonstrates readiness for deployment in streaming environments (Kafka, Spark Streaming)
- **Explainability:** rule_triggered column provides human-readable reason for every flagged event, critical for SOC analyst review
- **Offline / On-Premise:** No external API calls; fully deployable in air-gapped industrial networks
- **End-to-End Pipeline:** Covers ingestion, cleaning, ML detection, visualization, forecasting, and reporting in a single deployable application
- **Model Persistence:** All 4 models and scaler serialized together to ensure consistent feature scaling during inference without retraining

2.6 API Catalogue

This system is a standalone web application with no external REST APIs. Internal module interfaces:

Function Signature	Module	Returns
load_and_clean(source) -> DataFrame	preprocessing.py	Cleaned Pandas DataFrame
engineer_features(df) -> DataFrame	preprocessing.py	Feature-enriched DataFrame
cap_outliers(df, cols) -> DataFrame	preprocessing.py	Outlier-capped DataFrame
generate_synthetic_data(n: int) -> DataFrame	preprocessing.py	Synthetic network log DataFrame

Function Signature	Module	Returns
ThreatDetector.train(df: DataFrame)	detector.py	Fitted ThreatDetector instance
ThreatDetector.predict(df: DataFrame) -> DataFrame	detector.py	Annotated DataFrame with anomaly flags
ThreatDetector.score_threats(df: DataFrame) -> DataFrame	detector.py	Scored DataFrame with threat_level column
ThreatDetector.forecast_next_hour(df: DataFrame) -> (float, bool)	detector.py	Predicted bytes_sent, SPIKE_LIKELY flag
apply_heuristic_rules(df: DataFrame) -> DataFrame	detector.py	DataFrame with rule_triggered column
generate_pdf(df: DataFrame, stats: dict) -> bytes	report.py	FPDF binary content
export_csv(df: DataFrame) -> str	report.py	UTF-8 encoded CSV string

3. Data Design

3.1 Data Model

Field	Type	Description	Required
timestamp	DateTime	Timestamp of the network traffic event	Yes

Field	Type	Description	Required
device_ip	String	IP address of the source device	Yes
protocol	String	Network protocol: TCP, UDP, ICMP, etc.	Yes
bytes_sent	Float	Number of bytes transmitted from device	Yes
bytes_received	Float	Number of bytes received by device	Yes
connections	Integer	Number of concurrent connections	Yes
status	String	Traffic status label: Normal / Suspicious	Yes (imputed if missing)
hour	Integer	Derived: hour of day extracted from timestamp (0-23)	Engineered
day_of_week	Integer	Derived: day of week from timestamp (0=Monday)	Engineered
total_bytes	Float	Derived: bytes_sent + bytes_received	Engineered
protocol_encoded	Integer	Derived: numeric encoding of protocol (TCP=0, UDP=1, etc.)	Engineered
if_anomaly	Boolean	Computed: True if Isolation Forest flags as anomaly	Computed

Field	Type	Description	Required
lof_anomaly	Boolean	Computed: True if LOF flags as anomaly	Computed
km_anomaly	Boolean	Computed: True if K-Means distance > 90th percentile	Computed
rf_anomaly	Boolean	Computed: True if Random Forest predicts anomaly class	Computed
rule_triggered	String	Computed: name of heuristic rule triggered, or None	Computed
threat_score	Integer (0-100)	Computed: composite weighted threat severity score	Computed
threat_level	String	Computed: Critical / High / Medium / Low	Computed

3.2 Data Access Mechanism

- All data loaded into Pandas DataFrames; no SQL or NoSQL database used
- CSV files read via `pd.read_csv()` with UTF-8 encoding from data/ directory or Streamlit upload
- ML models and StandardScaler serialized via `joblib.dump()` to models/; loaded via `joblib.load()`
- Streamlit `session_state` persists processed DataFrames and model objects within a session
- Exported reports written via `DataFrame.to_csv()` and FPDF binary stream

3.3 Data Retention Policies

- **In-Memory Data:** All DataFrames exist only within the active Streamlit session; cleared on page reload
- **ML Models:** Serialized .pkl files in models/ persist across sessions; overwritten on each new training run
- **Exported Reports:** PDF and CSV files downloaded to the user's browser; not stored on the server
- **Synthetic Data:** Generated in-memory only; not written to disk
- **No user data is transmitted to external servers or cloud services**

3.4 Data Migration

This system requires no data migration as it is stateless beyond .pkl model files. If the feature engineering schema changes in a new version, existing .pkl files must be deleted and models retrained on new data to avoid feature mismatch errors during inference.

4. Interfaces

Interface	Type	Description
CSV File Upload	User Input	Streamlit file uploader in sidebar; accepts .csv network log files; falls back to synthetic data generation
Run Analysis Button	Trigger	Initiates full preprocessing + ML ensemble + scoring + forecasting pipeline
Live Mode Toggle	User Input	Checkbox enabling real-time simulation loop; generates and evaluates one new log entry every few seconds
Download PDF Report	Output	Streamlit download button for FPDF-generated threat summary report

Interface	Type	Description
Download CSV Export	Output	Streamlit download button for full processed DataFrame with all anomaly flags and threat scores
KPI Metric Cards	Output	Top-of-page display: Total Logs, Suspicious Events, Critical Threats, Peak Traffic Time
Security Analytics Tab	Output	7 Matplotlib/Seaborn charts: Traffic, Spike Detection, Heatmap, Scatter, Daily Trend, Protocol Pie, Clusters
Detailed Data & AI Insights Tab	Output	Per-model anomaly counts, next-hour forecast, SPIKE LIKELY warning, raw data table
models/ directory	File System	Read/write location for serialized .pkl model files
data/ directory	File System	Read location for input network log CSV files

5. State and Session Management

Streamlit re-executes the full script on every interaction. Critical state objects are persisted in `st.session_state` to prevent re-training and re-processing:

Session State Key	Type	Description
processed_df	Pandas DataFrame	Feature-engineered and scored network log DataFrame
threat_detector	ThreatDetector	Trained ThreatDetector instance with all 4 fitted ML models

Session State Key	Type	Description
forecast_value	Float	Predicted bytes_sent for next hour
spike_likely	Boolean	True if forecast exceeds 95th percentile of historical traffic
live_mode_active	Boolean	Flag controlling the live simulation loop state
simulation_buffer	Pandas DataFrame	Accumulates newly generated log entries during live simulation

Session state is reset on browser page reload. Trained model .pkl files on disk persist independently of session state.

6. Caching

- Streamlit's `@st.cache_data` can be applied to `load_and_clean()` and `engineer_features()` to cache results per unique file hash, avoiding reprocessing of unchanged uploads
- All 4 trained ML models and StandardScaler serialized to `models/*.pkl` via Joblib, acting as persistent disk cache across sessions to avoid retraining
- Chart figures can be stored in `session_state` to prevent regeneration on tab switches without data changes
- No external caching infrastructure (Redis, Memcached) required; the single-user, in-memory design does not necessitate distributed caching

7. Non-Functional Requirements

7.1 Security Aspects

Security Concern	Design Decision
Data Privacy	All processing is local; no network traffic data leaves the server; no external API calls
File Validation	Schema column check on upload; malformed files rejected with Streamlit error messages before processing begins
No Authentication Layer	Designed for controlled internal industrial network environments; authentication assumed at infrastructure level
Model File Security	Serialized .pkl files stored locally in models/; not exposed over any network endpoint
Pseudo-Label Transparency	Rule-based pseudo-labels used to train Random Forest are logged in rule_triggered column, ensuring labeling process is auditable
No Injection Risk	No SQL or shell operations; all data manipulation performed on in-memory Pandas objects

7.2 Performance Aspects

Performance Dimension	Design Decision
Training Speed	Four models trained once per analysis run; persisted to disk to eliminate retraining overhead on subsequent interactions
Inference Speed	All 4 model predictions run sequentially per batch; suitable for log batches up to ~100K records on standard hardware
Live Simulation Latency	One synthetic record generated and scored per cycle; near-zero latency for real-time UI updates
Visualization Rendering	Server-side Matplotlib/Seaborn chart generation; images served as static files, minimizing client browser load
PDF Generation	FPDF report scoped to top 20 critical threats only, bounding generation time regardless of total dataset size
Scalability	For multi-user SOC deployment, containerization via Docker and a job queue (Celery/RQ) for ML training are recommended
Memory Management	Outlier capping and feature selection (6 features) limit model memory footprint; large datasets should use chunked <code>pd.read_csv()</code>

8. References

- **Streamlit Documentation** — <https://docs.streamlit.io>
- **Pandas Documentation** — <https://pandas.pydata.org/docs>
- **NumPy Documentation** — <https://numpy.org/doc>
- **Scikit-Learn Documentation** — <https://scikit-learn.org/stable>
- **Matplotlib Documentation** — <https://matplotlib.org/stable/contents.html>
- **Seaborn Documentation** — <https://seaborn.pydata.org>
- **FPDF2 Documentation** — <https://py-pdf.github.io/fpdf2>
- **Joblib Documentation** — <https://joblib.readthedocs.io>
- **MITRE ATT&CK for ICS (Industrial Control Systems Threat Reference)** — <https://attack.mitre.org/matrices/ics>
- **Python Documentation** — <https://docs.python.org>